

Practical Exercise: Google BigQuery

Question 1

1
2 --1. WHERE Clause
3 --Q1. Filter all transactions that occurred in the year 2023. Expected output: All columns
4
5 SELECT *
6 FROM `coherent-parity-478118-m5.Sales.Records`
7 WHERE EXTRACT(YEAR FROM Date) = 2023;

✓ This query will process 72.69 KB when run.
Using on-demand processing quota

Query results [Save results](#) [Open in](#)

Job information Results Visualization JSON Execution details Execution graph

Row	Transaction ID	Date	Customer ID	Gender	Age	Product Category	Quantity	Price per Unit
1	191	2023-10-18	CUST191	Male	64	Beauty	1	
2	204	2023-09-28	CUST204	Male	39	Beauty	1	

Results per page: 50 1 – 50 of 998

Job history [Show](#)

Question 2

2 --Q2. Display all transactions where the Total Amount is more than the average Total Amount of the entire dataset. Expected output: All columns
3 SELECT
4 `Transaction ID`,
5 `Date`,
6 `Customer ID`,
7 `Gender`,
8 `Age`,
9 `Product Category`,
10 `Quantity`,
11 `Price per Unit`,
12 `Total Amount`
13 FROM `coherent-parity-478118-m5.Sales.Records`
14 WHERE `Total Amount` > (
15 SELECT AVG(`Total Amount`)
16 FROM `coherent-parity-478118-m5.Sales.Records`

✓ This script will process 106.29 KB when run.

Untitled query [Run](#) [Download](#) [Share](#) [Schedule](#) [Open in](#) [More](#) [Save](#)

✓ This query will process 72.69 KB when run.
Using on-demand processing quota

Query results [Save results](#) [Open in](#)

Job information Results Visualization JSON Execution details Execution graph

Row	Transaction ID	Date	Customer ID	Gender	Age	Product Category	Quantity	Price per Unit
1	21	2023-01-14	CUST021	Female	50	Beauty	1	
2	28	2023-04-23	CUST028	Female	43	Beauty	1	
3	128	2023-07-05	CUST128	Male	25	Beauty	1	
4	220	2023-03-03	CUST220	Male	64	Beauty	1	
5	238	2023-01-17	CUST238	Female	39	Beauty	1	
6	364	2023-08-23	CUST364	Female	19	Beauty	1	
7	408	2023-04-15	CUST408	Female	64	Beauty	1	

Results per page: 50 1 – 50 of 350

Question 3

```

30 -----
31 --3. Aggregate Functions
32 --Q3. Calculate the total revenue (sum of Total Amount). Expected output: Total_Revenue
33
34 SELECT SUM(`Total Amount`) AS `Total_Revenue`
35
36 FROM `coherent-parity-478118-m5.Sales.Records`
37

```

✓ This script will process 153.19 KB when run.

Using on-demand processing quota

Query results

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	Total_Revenue				
1	456000				

Question 4

```

38 -----
39 --4. DISTINCT
40 --Q4. Display all distinct Product Categories in the dataset. Expected output: Product_Category
41
42 SELECT DISTINCT `Product_Category` AS `Product_Category`
43
44 FROM `coherent-parity-478118-m5.Sales.Records`
45
46

```

✓ Query completed

Using on-demand processing quota

Query results

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	Product_Category				
1	Beauty				
2	Clothing				
3	Electronics				

Question 5

```

46 -----
47 --5. GROUP BY
48 --Q5. For each Product Category, calculate the total quantity sold. Expected output: Product_Category, Total_Quantity
49
50 SELECT DISTINCT `Product_Category` AS `Product_Category`,
51 | | | SUM(Quantity) AS `Total_Quantity`
52
53 FROM `coherent-parity-478118-m5.Sales.Records`
54 GROUP BY `Product_Category`
55
56

```

Syntax error: Expected end of input but got keyword SELECT at [42:1]

Using on-demand processing quota

Query results

Job information				Results	Visualization	JSON	Execution details	Execution graph
Row	Product_Category	Total_Quantity						
1	Beauty	771						
2	Clothing	894						
3	Electronics	849						

Question 6

```

57 --6. CASE Statement
58 --Q6. Create a column called Age_Group that classifies customers as 'Youth' (<30), 'Adult' (30-59), and 'Senior' (60+). Expected output: Customer_ID, Age, Age_Group
59
60 SELECT `Customer ID`,
61 | | | Age,
62 | | | CASE
63 | | | WHEN Age < 30 THEN 'Youth'
64 | | | WHEN Age BETWEEN 30 AND 59 THEN 'Adult'
65 | | | ELSE 'Senior'
66 | | | END AS `Age_Group`
67
68 FROM `coherent-parity-478118-m5.Sales.Records`

```

Syntax error: Expected end of input but got keyword SELECT at [42:1]

Using on-demand processing quota

Query results

[Save results](#) [Open in](#)

Job information				Results	Visualization	JSON	Execution details	Execution graph
Row	Customer ID	Age	Age_Group					
1	CUST191	64	Senior					
2	CUST204	39	Adult					
3	CUST230	54	Adult					
4	CUST232	43	Adult					

Results per page: 50 1 - 50 of 1000 |< < > >

Question 7

```
--7. Conditional Aggregation
--Q7. For each Gender, count how many high-value transactions occurred (where Total Amount > 500). Expected output: Gender, High_Value_Transactions

SELECT Gender,
       COUNT(*) AS `High_Value_Transactions`
FROM `coherent-parity-478118-m5.Sales.Records`
WHERE `Total Amount` > 500
GROUP BY Gender
```

Syntax error: Expected end of input but got keyword SELECT at [42:1]

Using on-demand processing quota

Query results

Save results

Job information Results Visualization JSON Execution details Execution graph

Row	Gender	High_Value_Transactions
1	Female	155
2	Male	144

Question 8

```
--Q8. For each Product Category, show only those categories where the total revenue exceeds 5,000. Expected output: Product_Category, Total_Revenue

SELECT `Product Category`,
       SUM(`Total Amount`) AS `Total_Revenue`
FROM `coherent-parity-478118-m5.Sales.Records`
GROUP BY `Product Category`
HAVING `Total_Revenue` > 5000;
```

3 --Q8. For each Product Category, show only those categories where the total revenue exceeds 5,000. Expected output: Product_Category, Total_Revenue

✓ This query will process 17.98 KB when run.

Using on-demand processing quota

Query results

Save results Open in

Job information Results Visualization JSON Execution details Execution graph

Row	Product Category	Total_Revenue
1	Beauty	143515
2	Clothing	155580
3	Electronics	156905

Question 9

```
--9. Calculated Fields
--Q9. Display a new column called Unit_Cost_Category that labels a transaction as: - 'Cheap' if Price per Unit < 50 - 'Moderate' if Price per Unit between 50 and 200 - 'Expensive' if Price per Unit > 200 Expected output: Transaction_ID, Price_per_Unit, Unit_Cost_Category

SELECT `Transaction ID`,
       `Price per Unit`,
       CASE
         WHEN `Price per Unit` > 200 THEN 'Expensive'
         WHEN `Price per Unit` BETWEEN 50 AND 200 THEN 'Moderate'
         ELSE 'Cheap'
       END AS `Unit_Cost_Category`
FROM `coherent-parity-478118-m5.Sales.Records`
```

✓ This script will process 33.61 KB when run.

Using on-demand processing quota

Query results

[Save results](#)

[Open in](#)



Job information **Results** Visualization JSON Execution details Execution graph

Row	Transaction ID	Price per Unit	Unit_Cost_Category
42	207	25	Cheap
43	273	50	Moderate
44	349	50	Moderate
45	391	25	Cheap
46	450	25	Cheap
47	460	50	Moderate

Results per page: 50

1 – 50 of 1000



Job history

[Show](#)